Implementation of Simple Clause

```
import itertools


# A tiny Prolog-style engine.  Variables start with a capital letter (X, Y),
# constants are lowercase (mary, wine).  A clause is (head, [body goals]);
# a fact is a clause with an empty body.
KB = []
_ids = itertools.count()


def fact(pred, *args):
    KB.append(((pred, args), []))


def g(pred, *args):
    return (pred, args)


def rule(head, *body):
    KB.append((head, list(body)))


def kb_is_var(t):
    return t[:1].isupper()


def kb_walk(t, theta):
    while kb_is_var(t) and t in theta:
        t = theta[t]
    return t


def kb_unify(a, b, theta):
    theta = dict(theta)
    for x, y in zip(a, b):
        x, y = kb_walk(x, theta), kb_walk(y, theta)
        if x == y:
            continue
        if kb_is_var(x):
            theta[x] = y
        elif kb_is_var(y):
            theta[y] = x
        else:
            return None
    return theta


def rename_clause(head, body):
    n = str(next(_ids))
    r = lambda g: (g[0], tuple(a + n if kb_is_var(a) else a for a in g[1]))
    return r(head), [r(g) for g in body]


def solve(goals, theta, depth=0):
    """goals = list of (goal, ancestors). Yields every substitution proving all goals."""
    if not goals or depth > 12:
        if not goals:
            yield theta
        return
    (goal, ancestors), rest = goals[0], goals[1:]
    current = (goal[0], tuple(kb_walk(a, theta) for a in goal[1]))
    if current in ancestors:                    # loop check
        return
    for head, body in KB:
        head, body = rename_clause(head, body)
        if head[0] == goal[0] and len(head[1]) == len(goal[1]):
            t2 = kb_unify(goal[1], head[1], theta)
            if t2 is not None:
                sub = [(g, ancestors + (current,)) for g in body]
                yield from solve(sub + rest, t2, depth + 1)
def query(pred, *args):
```

Implementation of Simple Clause

```
    print(f"| ?- {pred}({','.join(args)}).")
    qvars = [a for a in args if kb_is_var(a)]
    answers = []
    for th in solve([((pred, args), ())], {}):
        ans = tuple(kb_walk(v, th) for v in qvars)
        if ans not in answers:
            answers.append(ans)
    if not answers:
        print("no.")
    elif not qvars:
        print("yes.")
    else:
        for ans in answers:
            print(", ".join(f"{v} = {a}" for v, a in zip(qvars, ans)))


def program6():
    print("\n=== 6. SIMPLE CLAUSE ===")
    fact("likes", "mary", "food")
    fact("likes", "mary", "wine")
    fact("likes", "john", "wine")
    fact("likes", "john", "mary")
    query("likes", "mary", "food")
    query("likes", "john", "wine")
    query("likes", "john", "food")

    print("\n-- Adding the rules --")
    print("1. John likes anything that Mary likes")
    rule(g("likes", "john", "X"), g("likes", "mary", "X"))
    print("2. John likes anyone who likes wine")
    rule(g("likes", "john", "X"), g("likes", "X", "wine"))
    print("3. John likes anyone who likes themselves")
    rule(g("likes", "john", "X"), g("likes", "X", "X"))
    print()
    query("likes", "john", "food")
    query("likes", "john", "X")


if __name__ == "__main__":
    program6()
```